RISC-V 시뮬레이터 구현 프로젝트 보고서

항목	내용
팀	11조
팀원 (4명)	성주원(【학번】), 김도윤(【학번】), 송윤재(【학번】), 정재운(【학번】)
구현 언어	Python 3 (3.8 이상, 표준 라이브러리만 사용)
실행 방법	`python3 sim.py <text.bin> <data.bin>` (빌드 과정 없음)
제출일	【YYYY-MM-DD】

> **작성 안내 (제출 전 이 상자를 지우세요)** - 【작성 필요】 표시는 팀만 알 수 있는 정보입니다. 사실대로 채우고, 특히 11장 AI 활용 내역은 실제 사용 내용과 일치해야 합니다(명세 9.2절, 허위 기재 금지).

요약

명세서가 지정한 RV32I 14개 명령어를 실행하는 ISA 수준 기능 시뮬레이터를 Python 3 표준 라이브러리만으로 구현했다. 시뮬레이터 본체는 `sim.py`와 `rvsim/` 패키지의 네 모듈(`bits`, `memory`, `decoder`, `cpu`), 합계 약 480줄이다. 검증은 다섯 단계로 했다.

검증 단계	방법	결과
1. 종단 간 테스트	손으로 기대 결과를 도출한 테스트 40개와 CLI 오류 4개를 별도 프로세스로 실행하고, 표준 출력과 `data.out`을 바이트 단위로 비교	44/44 통과
2. 단위 테스트	즉시값 경계, 해독 규칙(무작위 명령어 값 200,000개 포함), 메모리, 실행 한도 경계	30/30 통과
3. 변이 테스트	흔한 구현 실수 29종을 일부러 넣었을 때 테스트가 잡아내는지 확인	29/29 검출
4. 참조 실행기 대조	Unicorn Engine(테스트 전용)과 비교: 손 계산 기대값 33개, 무작위 프로그램 23,000개	전부 일치
5. 새 환경 실행	Docker 리눅스 컨테이너(Python 3.8.20 / 3.13.18)에서 README 명령만으로 실행	전부 통과

1. 지원 범위와 실행 방법 (①)

1.1 지원 명령어

명령어	형식	opcode	funct3	funct7	동작
add	R	0110011	000	0000000	rd = rs1 + rs2 (하위 32비트)
sub	R	0110011	000	0100000	rd = rs1 - rs2 (하위 32비트)
and	R	0110011	111	0000000	rd = rs1 & rs2
or	R	0110011	110	0000000	rd = rs1 | rs2
addi	I	0010011	000	-	rd = rs1 + sext(imm12)

명령어	형식	opcode	funct3	funct7	동작
andi	I	0010011	111	-	rd = rs1 & sext(imm12)
ori	I	0010011	110	-	rd = rs1 | sext(imm12)
lui	U	0110111	-	-	rd = imm20 << 12
lw	I	0000011	010	-	rd = M[rs1 + sext(imm12)] (4바이트, little-endian)
sw	S	0100011	010	-	M[rs1 + sext(imm12)] = rs2
beq	B	1100011	000	-	rs1 == rs2 이면 PC = PC + sext(imm13)
bne	B	1100011	001	-	rs1 != rs2 이면 분기
blt	B	1100011	100	-	(부호 있는) rs1 < rs2 이면 분기
bge	B	1100011	101	-	(부호 있는) rs1 >= rs2 이면 분기

이 표에 없는 opcode·funct3·funct7 조합은 모두 **invalid instruction**이며, fetch하면 실행하지 않고 종료한다. 압축 명령어, 시스템 호출, 예외·인터럽트, 자기 수정 코드는 명세에 따라 지원하지 않는다.

1.2 실행 방법

```
python3 sim.py <text.bin 경로> <data.bin 경로>     # Windows 는 python 또는 py
```

- 인수는 정확히 두 개(text, data 순서)이며, 빌드 과정이 없다. 네트워크·대화형 입력을 쓰지 않는다.
- 정상 종료하면 현재 작업 디렉터리에 `data.out`(1,024 바이트)을 덮어쓰고, 표준 출력에 한 줄을 쓴다. 예: `PC 0x00040014 invalid instruction 0x00000000`
- 종료 코드: 0 정상 / 1 인수·파일 입출력 오류 / 2 실행 한도 초과 / 3 메모리 범위·정렬 위반(명세 범위 밖 입력)

1.3 기능별 구현 위치표

기능	파일	함수·위치	설명
명령어 인출(fetch)	`rvsim/memory.py`	`Memory.fetch`	text 영역 범위·4바이트 정렬을 확인하고 4바이트를 little-endian으로 조합
	`rvsim/cpu.py`	`CPU.run`	`fetch_pc = self.pc`로 fetch 당시 주소를 보존하고 fetch → decode → execute를 반복
R형 처리	`rvsim/decoder.py`	`decode`의 `OPCODE_OP` 분기, `R_TYPE_TABLE`	(funct3, funct7) 쌍으로 add/sub/and/or를 판별
	`rvsim/cpu.py`	`CPU.execute` (add, sub, and, or)	연산 후 `write_reg`에서 하위 32비트를 저장
I형 처리	`rvsim/decoder.py`	`decode`의 `OPCODE_OP_IMM`·`OPCODE_LOAD` 분기, `imm_i`	funct3 판별, 12비트 즉시값 부호 확장
	`rvsim/cpu.py`	`CPU.execute` (addi, andi, ori, lw)	andi/ori는 `to_u32(imm)`로 32비트 마스크를 만든 뒤 연산
S형 처리	`rvsim/decoder.py`	`decode`의 `OPCODE_STORE` 분기, `imm_s`	imm[11:5]=bit31:25, imm[4:0]=bit11:7을 결합하고 부호 확장

기능	파일	함수·위치	설명
B형 처리	`rvsim/decoder.py`	`decode`의 `OPCODE_BRANCH` 분기, `imm_b`	imm[12|10:5|4:1|11]을 재조립하고 13비트 부호 확장
U형 처리	`rvsim/decoder.py`	`decode`의 `OPCODE_LUI` 분기, `imm_u`	`word & 0xFFFFF000`
lw·sw	`rvsim/cpu.py`	`CPU.execute` (lw, sw)	유효 주소 = `to_u32(rs1 + imm)`
	`rvsim/memory.py`	`Memory.load_word`, `Memory.store_word`, `Memory._data_offset`, `read_word_le`, `write_word_le`	데이터 영역 범위·정렬 확인, little-endian 4바이트 읽기·쓰기
분기	`rvsim/cpu.py`	`CPU.execute` (beq, bne, blt, bge)	목표 = 분기 명령어 주소 + imm. blt/bge는 `to_signed32`로 부호 있는 비교
invalid 판별	`rvsim/decoder.py`	`decode`	표에 없는 opcode·funct3·funct7 조합이면 `None` 반환
	`rvsim/cpu.py`	`CPU.run`	`inst is None`이면 실행하지 않고 `(fetch_pc, word)` 반환
실행 한도	`rvsim/cpu.py`	`MAX_INSTRUCTIONS`, `CPU.run`, `ExecutionLimitExceeded`	invalid가 아니고 이미 1,000,000개를 실행했으면 비정상 종료
data.out 생성	`rvsim/memory.py`	`write_data_out`	`"wb"` 모드로 1,024 바이트를 그대로 저장(덮어쓰기)
	`sim.py`	`main`	종료 처리 순서: data.out 저장 → 표준 출력
표준 출력	`sim.py`	`format_termination_line`, `main`	`"PC 0x%08x invalid instruction 0x%08x\n"`을 바이트로 출력
32비트·부호	`rvsim/bits.py`	`to_u32`, `to_signed32`, `sign_extend`, `get_bits`	32비트 자르기, 2의 보수 해석, 부호 확장, 비트 필드 추출
x0 고정	`rvsim/cpu.py`	`CPU.write_reg`	`index != 0`일 때만 쓰기
입력 적재	`rvsim/memory.py`	`read_binary_file`, `Memory.__init__`	`"rb"`로 읽고, data는 1,024 바이트를 0으로 초기화한 뒤 복사

2. PC·레지스터·메모리 표현 (②)

PC - `CPU.pc`(Python 정수)이며 초기값은 `TEXT_BASE = 0x00040000`이다. 항상 0 ~ 0xFFFFFFFF 범위로 유지한다(`CPU.execute`가 `to_u32(next_pc)`를 반환).

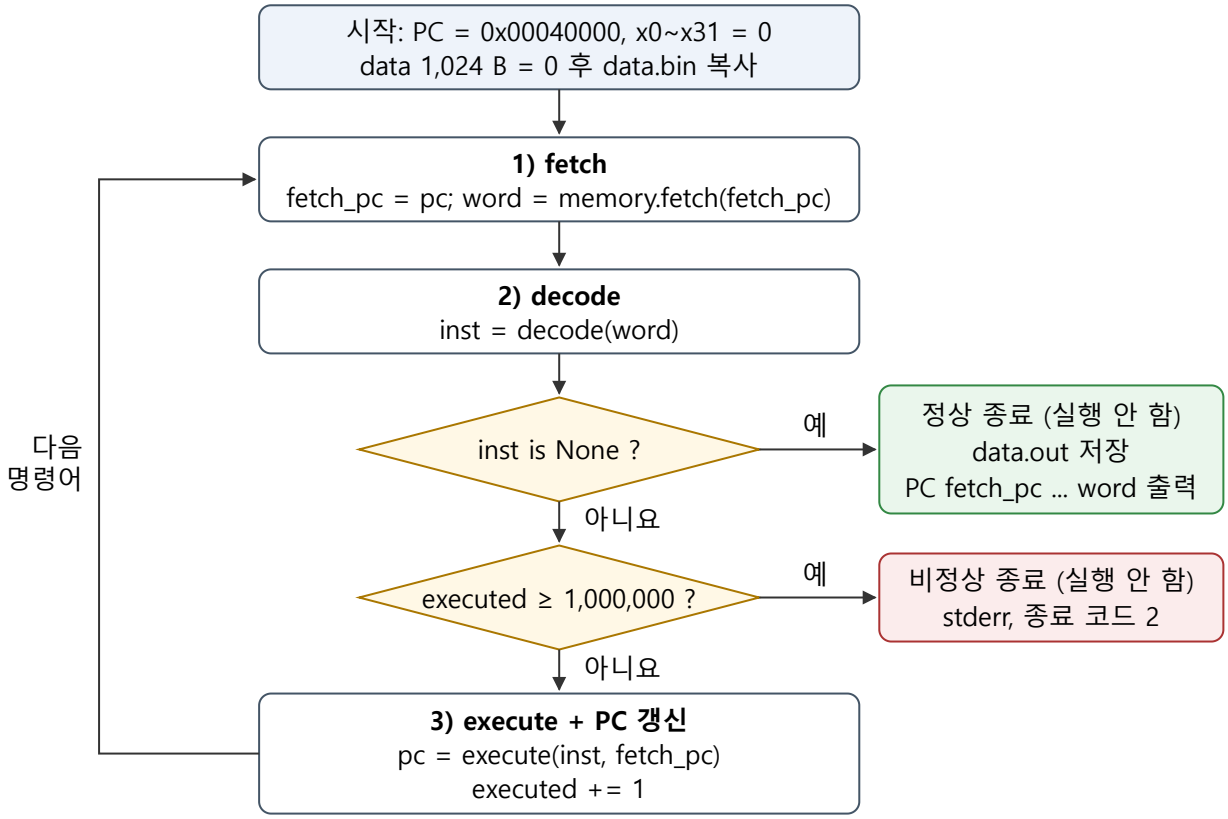
레지스터 - `CPU.regs = [0] * 32`, 즉 x0~x31을 모두 0으로 시작하며 스택 포인터를 따로 초기화하지 않는다. 각 원소는 **부호 없는 32비트 비트 패턴**(0 ~ 0xFFFFFFFF)으로 보관한다. 예를 들어 -1은 `0xFFFFFFFF`로 저장한다. 모든 쓰기는 `CPU.write_reg`를 거친다. 이 함수는 ① `index == 0`이면 무시하고 ② 그 밖에는 `to_u32(value)`로 하위 32비트만 저장한다. x0에는 아무것도 쓰이지 않으므로 `self.regs[0]`을 읽으면 항상 0이다.

메모리 - 명세 6절의 "메모리 분리"에 따라 두 영역을 따로 둔다.

영역	주소 범위	표현	접근
text	0x00040000 ~ 0x00040000 + len(text.bin) - 1	`Memory.text` (`bytes`, 수정 불가)	`Memory.fetch`만
data	0x10000000 ~ 0x100003FF (1,024 B)	`Memory.data` (`bytearray(1024)`)	`Memory.load_word`, `Memory.store_word`만

주소를 배열 인덱스로 바꾸는 식은 `offset = addr - 영역 시작 주소`이다. 접근할 때마다 `addr % 4 == 0`이고 `0 <= offset`, `offset + 4 <= 영역 크기`인지 검사한다. text를 `bytes`로 두었고 `store_word`는 데이터 영역만 허용하므로, text segment 쓰기는 구조적으로 불가능하다(`sw`로 0x00040000에 쓰면 종료 코드 3).

3. fetch / decode / execute 및 PC 갱신 흐름 (③)



`CPU.run`의 루프(위 그림)는 다음과 같다.

```
while True:
    fetch_pc = self.pc                      # fetch 당시 주소를 따로 보존
    word = self.memory.fetch(fetch_pc)      # 1) fetch
    inst = decode(word)                     # 2) decode
    if inst is None:                        # 지원하지 않는 명령어 -> 실행 없이 종료
        return fetch_pc, word
    if self.executed >= limit:              # 이미 한도만큼 실행했는데 또 정상 명령어
        raise ExecutionLimitExceeded(fetch_pc, word)
    self.pc = self.execute(inst, fetch_pc)  # 3) execute + PC 갱신
    self.executed += 1
```

PC 갱신 규칙 (`CPU.execute`)

- 모든 명령어에서 먼저 `next_pc = pc + 4`로 두고, 분기 조건이 참이면 `next_pc = pc + imm`으로 바꾼다. 여기서 `pc`는 **분기 명령어 자신의 주소**(fetch_pc)다. PC+4를 기준으로 하지 않는다.
- `execute`는 `to_u32(next_pc)`를 반환하고, `run`이 그 값을 `self.pc`에 넣는다. PC를 미리 증가시키지 않으므로 종료 시 출력하는 PC는 자연스럽게 fetch 당시 주소가 된다. 게다가 `fetch_pc` 변수를 따로 두어 이를 명시적으로 보장한다.
- **invalid 판별을 실행 한도 검사보다 먼저** 한다. 그래야 "정확히 1,000,000개를 실행한 뒤 invalid를 만나면 정상 종료"(명세 6절)가 성립한다. 두 검사의 순서를 바꾸면 33번 테스트가 실패한다(7.4절 변이 테스트).

4. 즉시값·부호·32비트 처리 (④)

값 표현 원칙. Python의 `int`는 크기 제한이 없고 음수를 그대로 표현한다. 따라서 명세의 "호스트 언어의 정수 크기·부호 규칙이 시뮬레이터의 의미를 바꾸어서는 안 된다"를 지키려면 명시적으로 처리해야 한다.

위치	처리	근거 함수
레지스터 쓰기	하위 32비트만 저장 (덧셈 자리올림 버림, 음수는 2의 보수 패턴)	`bits.to_u32` ← `CPU.write_reg`
즉시값	해독할 때 부호 확장하여 Python 음수/양수로 보관	`bits.sign_extend` ← `imm_i`, `imm_s`, `imm_b`
andi/ori	부호 확장된 imm을 32비트 패턴으로 바꾼 뒤 연산 (-1 → 0xFFFFFFFF)	`to_u32(imm)` in `CPU.execute`
blt/bge	두 레지스터 값을 2의 보수 정수로 해석해 비교	`bits.to_signed32`
beq/bne	비트 패턴 그대로 비교 (부호 무관)	-
주소·PC	`to_u32(rs1 + imm)`, `to_u32(next_pc)`	`CPU.execute`

즉시값 형식 (`rvsim/decoder.py`)

형식	함수	비트 재조립	부호 확장 폭	범위
I	`imm_i`	imm[11:0] = inst[31:20]	12	-2048 ~ 2047
S	`imm_s`	imm[11:5] = inst[31:25], imm[4:0] = inst[11:7]	12	-2048 ~ 2047
B	`imm_b`	imm[12] = inst[31], imm[11] = inst[7], imm[10:5] = inst[30:25], imm[4:1] = inst[11:8], imm[0] = 0	13	-4096 ~ 4094 (짝수)
U	`imm_u`	imm[31:12] = inst[31:12], 하위 12비트 0	(32비트 그대로)	-

예시

- `addi x2, x0, -1` = `0xfff00113`이다. inst[31:20] = `0xfff`를 12비트 부호 확장하면 -1이고, x2 = `to_u32(0 + -1)` = `0xffffffff`이다.
- `blt x2, x3` (x2 = `0xffffffff`, x3 = `0x00000001`): `to_signed32(0xffffffff)` = -1 < 1 이므로 **분기한다**. 부호 없는 비교라면 4294967295 < 1이 거짓이어서 분기하지 않는다(테스트 14).
- `andi x7, x2, -1`: imm = -1 → `to_u32(-1)` = `0xffffffff` → x2가 그대로 남는다. 하위 12비트만 쓰면 `0xfff`가 되어 틀린다(테스트 06).
- `lui x4, 0x12346` 후 `addi x4, x4, -1` → `0x12345fff`. addi 즉시값이 부호 확장되므로, 하위 12비트가 0x800 이상인 상수는 lui에 1을 더해 만든다(테스트 07).

Python 고유의 함정 (검증 중 발견, 8장 참조). Python의 `&`, `>>`는 음수를 무한 길이의 2의 보수처럼 다룬다. 그래서 레지스터를 32비트로 자르지 않아도 `sw`가 바이트를 뽑을 때(`value & 0xFF`) 우연히 올바른 바이트가 나온다. 이 버그는 레지스터끼리 `beq/bne`로 비교할 때만 드러난다(예: -1과 `0xffffffff`). 그래서 모든 레지스터 쓰기를 `write_reg` 한 곳에 모아 자르고, 테스트 08로 확인한다.

5. 바이너리 입출력 (⑤)

항목	구현	위치
입력 읽기	`open(path, "rb")`로 헤더 없는 바이너리를 그대로 읽는다(ASCII·16진수 텍스트로 해석하지 않음)	`memory.read_binary_file`
적재	text.bin 첫 바이트 → 0x00040000. data는 `bytearray(1024)`(전체 0)를 만든 뒤 data.bin을 앞에서부터 복사(0~1,024 B)	`Memory.__init__`
4바이트 → 값	`b0 | (b1 << 8) | (b2 << 16) | (b3 << 24)` (명세 4절 식 그대로)	`memory.read_word_le`
값 → 4바이트	하위 바이트부터 `value & 0xFF`, `(value >> 8) & 0xFF`, …	`memory.write_word_le`
data.out	`open("data.out", "wb")`로 0x10000000 ~ 0x100003FF를 주소 순서대로 1,024 바이트 기록. 헤더·줄바꿈 없음, 기존 파일 덮어쓰기	`memory.write_data_out`
표준 출력	`sys.stdout.buffer.write(line.encode("ascii"))`	`sim.main`

little-endian 예. 파일 바이트 `78 56 34 12`는 `0x78 | 0x56<<8 | 0x34<<16 | 0x12<<24 = 0x12345678`이다. 명세 예시의 첫 명령어 바이트 `b7 00 00 10`은 `0x100000b7`(lui x1, 0x10000)이다. 바이트를 파일 순서대로 이어 붙인 문자열 `b7000010`과 혼동하지 않는다.

바이너리 모드를 쓰는 이유. Windows에서 텍스트 모드로 쓰면 줄바꿈 변환이 일어난다.

- data.out의 `0x0A` 바이트가 `0x0D 0x0A`로 바뀌어 파일이 1,024 바이트보다 커진다.
- 표준 출력의 `\n`도 `\r\n`이 된다.

그래서 data.out은 `"wb"`, 표준 출력은 `sys.stdout.buffer`(바이트 스트림)로 쓴다. 이를 확인하기 위해 테스트 02·11·13에 `0x0A`·`0x0D` 바이트가 들어 있고, 텍스트 모드 변이를 Windows에서 실제로 검출했다(7.4절).

6. invalid 판별과 종료 처리 (⑥)

판별 (`decoder.decode`). opcode(bit 6:0)로 먼저 나누고, 각 opcode마다 허용하는 funct3(R형은 funct7까지)를 표로 확인한다.

opcode	허용 조합	표에 없는 예 → invalid
0110011 (R)	(000,0000000) add, (000,0100000) sub, (111,0000000) and, (110,0000000) or	mul(funct7=0000001), xor(funct3=100), sra(101,0100000)
0010011 (I 연산)	funct3 000 addi, 111 andi, 110 ori	slli(001), slti(010), xori(100)
0000011 (load)	funct3 010 lw	lb(000), lh(001), lbu(100)
0100011 (store)	funct3 010 sw	sb(000), sh(001)
1100011 (branch)	funct3 000 beq, 001 bne, 100 blt, 101 bge	bltu(110), bgeu(111)
0110111 (lui)	항상	-

opcode	허용 조합	표에 없는 예 → invalid
그 밖의 모든 opcode	없음	jal(1101111), ecall(1110011), 0x00000000, 0xffffffff

`decode`는 유효하면 `Instruction`을, 아니면 `None`을 반환한다. `test_units.py`는 무작위 32비트 값 200,000개를 따로 작성한 판정식과 비교하고, 모든 opcode(128개)와 각 opcode의 funct3(8개), R형 funct7(128개)를 빠짐없이 검사한다.

정상 종료 순서.

1. `CPU.run`이 `inst is None`을 보면 그 명령어를 **실행하지 않고** `(fetch_pc, word)`를 반환한다. 따라서 invalid 뒤의 명령어 효과는 반영되지 않는다(테스트 22~32).
2. `sim.main`이 `write_data_out("data.out", memory)`로 1,024 바이트를 저장한다.
3. `format_termination_line(pc, word)`로 `PC 0x%08x invalid instruction 0x%08x\n`을 만들어 표준 출력에 쓰고, 종료 코드 0을 반환한다.

실행 한도 (명세 6절). `executed`는 실행을 마친 정상 명령어 수다. 다음 fetch가 invalid면 위의 정상 종료로 간다. 지원 명령어인데 `executed >= 1,000,000`이면 `ExecutionLimitExceeded`를 일으킨다. 이때 `sim.main`은 표준 오류에 PC와 명령어 값을 쓰고 종료 코드 2를 반환한다. 표준 출력과 data.out은 만들지 않는다(명세에 형식이 없어 AI 제안을 팀이 채택).

명세 범위 밖 입력 (팀 결정). 다음 경우는 표준 오류 메시지와 함께 종료 코드 3을 반환한다.

- text 영역 밖이나 비정렬 주소에서 fetch
- 데이터 영역 밖이나 비정렬 주소에 lw/sw (text 쓰기 포함)

text.bin 크기가 4의 배수가 아니거나 data.bin이 1,024 바이트를 넘으면 경고만 하고 계속 실행한다.

7. 테스트와 기대 결과의 도출 (⑦)

7.1 테스트 구성

파일	역할
`tests/asm.py`	테스트 입력을 만드는 미니 어셈블러. **시뮬레이터의 해독기와 코드를 공유하지 않는 독립 인코더**다. 같은 실수가 양쪽에 숨으면 검증이 되지 않기 때문이다.
`tests/cases.py`	40개 케이스: 어셈블리, data.bin, **손으로 계산한 기대 결과**와 근거 주석
`tests/gen_tests.py`	`tests/cases/<이름>/`에 text.bin, data.bin, expected_stdout.txt, expected_data.out, expected_exit_code.txt, program.s를 만든다. 생성 전에 교차 확인도 한다: (1) 01번 text.bin이 명세 7절 바이트열과 같은지 (2) 손으로 계산한 invalid 명령어 값이 어셈블러 결과와 같은지
`tests/run_tests.py`	실제 채점처럼 **별도 프로세스**로 실행한다. 새 임시 디렉터리를 작업 디렉터리로 쓰고, 쓰레기 data.out(2,048 B)을 미리 둬서 덮어쓰기까지 확인한다. 종료 코드, 표준 출력 바이트, data.out 크기·바이트를 비교한다.
`tests/test_units.py`	단위 테스트 30개
`tests/mutation_test.py`	변이 테스트 29종
`tests/diff_ref_unicorn.py`	참조 실행기 대조 (테스트 전용)

7.2 테스트 범주

범주	케이스	확인 내용
기본 입출력	01~02	명세 예시 재현, 첫 명령어가 invalid일 때 data.bin → data.out 그대로
레지스터·산술·논리	03~08	x0 고정, 오버플로 자르기, 즉시값 경계(-2048/2047), andi/ori 부호 확장, lui+addi 상수, 32비트 비교
메모리·바이트 순서	09~12	little-endian, 첫/마지막 워드, 음수·큰 오프셋, 빈 data.bin, 4의 배수가 아닌 data.bin
분기·프로그램	13~21	beq/bne/blt/bge 참·거짓, 부호 비교, B형 즉시값 비트(+2048, +2032, -4096), 건너뛴 invalid, 루프, 피보나치, 부호 곱셈, 버블 정렬, x1~x31 필드
invalid 판별	22~32	mul, sra, and+funct7, xor, slli, lb, sh, bltu, jal, ecall, 0xffffffff
실행 한도	33~36	정확히 1,000,000개 → 정상, 1,000,001개 필요 → 비정상, 1,000,002개 필요, 무한 루프
범위 밖 입력	37~40	데이터 영역 밖 lw, 비정렬 sw, text 밖 분기, text 쓰기 → 종료 코드 3
CLI	4개	인수 0/1/3개, 없는 파일 → 종료 코드 1, 표준 출력 없음

전체 목록은 부록 A에 있다.

7.3 기대 결과 도출 예

(1) 명세 예시 (01). 각 명령어를 순서대로 계산한다.

- lui x1, 0x10000 → x1 = `0x10000000`
- x2 = 7, x3 = 5, x4 = 12
- sw → [0x10000000] = `0x0000000c` → 바이트 `0c 00 00 00`
- 0x00040014의 `0x00000000`은 opcode 0000000이므로 invalid

따라서 기대 출력은 `PC 0x00040014 invalid instruction 0x00000000`이다. 생성기가 text.bin을 명세의 `b7 00 00 10 13 01 70 00 ...`과 바이트 단위로 대조한다.

(2) 부호 비교 (14). x2 = -1(`0xffffffff`), x3 = 1, x4 = INT_MIN, x5 = INT_MAX이다. 분기 8개가 각각 다음 `ori`를 건너뛰는지 따진다.

분기	비교	결과	x10에 켜지는 비트
blt -1, 1	참	분기	-
bge -1, 1	거짓	진행	2
blt 1, -1	거짓	진행	4
bge 1, -1	참	분기	-
blt MIN, MAX	참	분기	-
bge MIN, MAX	거짓	진행	32
bge 1, 1	참	분기	-
blt 1, 1	거짓	진행	128

x10 = 2 + 4 + 32 + 128 = **0xa6**이다. 부호 없는 비교로 잘못 구현하면 1 + 8 + 16 + 128 = 0x99가 된다.

(3) 실행 한도 (33, 34). 33번은 lui·addi로 x1 = 0x7a11f = 499,999를 만들고, `addi -1; bne` 루프를 499,999번 돈다. 실행 수는 2 + 2 × 499,999 = **1,000,000**이고, 그다음 fetch(0x00040010)가 invalid이므로 정상 종료한다.

34번은 앞에 `nop`을 하나 더 넣어 1,000,001개가 필요하다. 1,000,000번째로 마지막 반복의 addi를 실행한 뒤, 다음 fetch(bne)가 지원 명령어이므로 비정상 종료(종료 코드 2)한다.

(4) invalid 명령어 값 (22). `mul x5, x2, x3`의 필드는 funct7 = 0000001, rs2 = 3, rs1 = 2, funct3 = 000, rd = 5, opcode = 0110011이다.

```
  (1 << 25) + (3 << 20) + (2 << 15) + (5 << 7) + 0x33
= 0x02000000 + 0x00300000 + 0x00010000 + 0x00000280 + 0x33
= 0x023102b3
```

손으로 계산한 이 값이 어셈블러 결과와 같은지 생성기가 확인한다.

7.4 결과

검증	결과
`python tests/run_tests.py` (Windows 11, Python 3.10.0)	44/44 통과
`python tests/test_units.py`	30/30 통과
`python tests/mutation_test.py`	29/29 변이 검출 (부록 B)
Unicorn 대조: 정상 종료 케이스 33개의 손 계산 data.out	33/33 일치
Unicorn 대조: 무작위 프로그램 (seed 1: 3,000개, seed 2026: 20,000개)	23,000/23,000 일치. 레지스터 32개, data 1,024 B, 종료 PC를 비교
Docker `python:3.8-slim`, `python:3.13-slim` (Linux, 새 환경)	단위 30/30, 종단 간 44/44 통과
1,000,000개 실행 시간 (개발 PC)	약 2.5초

무작위 프로그램은 다음과 같이 만들었다.

- 레지스터 초기값은 무작위로 하되 0, -1, INT_MIN 같은 특수값을 섞었다.
- R/I/U형 연산, lw/sw(데이터 영역 전체, 양수·음수 오프셋), 앞으로 가는 분기, 정해진 횟수의 루프 블록을 넣었다.
- 프로그램당 평균 약 118개 명령어를 실행했고, 실행된 분기의 49%가 실제로 분기했다.
- 이 대조가 버그를 잡아내는지도 확인했다. blt/bge 부호 처리 버그와 32비트 자르기 누락 버그를 주입하면 불일치를 보고했다.

8. 발견한 오류와 수정 근거 (⑧)

#	발견 경위	내용	수정·근거
1	변이 테스트 1차 실행 (27/29 검출)	"레지스터를 32비트로 자르지 않음" 변이가 **어떤 종단 간 테스트로도 검출되지 않았다.** Python의 비트 연산이 음수를 2의 보수처럼 다뤄서 `sw` 결과 바이트는 우연히 맞았다.	레지스터끼리 비교하는 테스트 **08_wrap_compare**를 추가했다(addi로 만든 -1 대 lw로 읽은 0xffffffff, 0xffffffff+1 대 0). 추가 후 이 변이를 검출했다.
2	같은 실행	"실행 한도 비교를 `>`로" 변이가 검출되지 않았다. 1,000,002개가 필요한 35번 테스트는 `>`여도 비정상 종료하기 때문이다.	정확히 1,000,001개가 필요한 경계 테스트 **34_limit_plus_one**을 추가했다. 33번(1,000,000 → 정상)과 함께 경계 양쪽을 모두 확인한다. 추가 후 29/29 검출.

#	발견 경위	내용	수정·근거
3	설계 단계 위험 분석	Windows 텍스트 모드의 줄바꿈 변환 때문에 data.out 크기와 표준 출력 줄바꿈이 달라질 위험이 있었다.	data.out은 `"wb"`, 표준 출력은 `sys.stdout.buffer`로 쓴다. 텍스트 모드 변이 2종을 Windows에서 검출하는 것을 확인했다. 리눅스에서는 줄바꿈 변환이 없어 이 변이가 드러나지 않는다(9장).

시뮬레이터 본체(`sim.py`, `rvsim/`)에서는 위 테스트들로 검출된 기능 오류가 없었다. 1·2번은 **테스트 모음의 빈틈**을 찾아 보강한 사례다. 【팀이 직접 검토·실행하면서 발견한 오류가 있으면 이 표에 추가한다】

9. 알려진 제한사항 (⑨)

- **속도:** Python 인터프리터라서 1,000,000개 실행에 개발 PC 기준 약 2.5초가 걸린다(명세상 속도는 평가 항목이 아님). text segment는 쓰기 불가이므로 해독 결과를 캐시하면 빨라질 수 있다. 다만 fetch → decode → execute 흐름을 코드에 그대로 보이기 위해 넣지 않았다.
- **지원 범위:** 14개 명령어만 지원한다. jal/jalr/auipc/shift/byte·halfword 메모리 접근 등은 invalid로 종료한다.
- **데이터 메모리:** 0x10000000 ~ 0x100003FF 1,024 바이트만 지원한다. 범위 밖이나 비정렬 lw/sw는 종료 코드 3으로 끝난다(명세가 보장하는 채점 입력에서는 발생하지 않음).
- **비정상 종료 형식:** 실행 한도 초과·범위 위반 때의 출력(표준 출력 없음, data.out 미생성, 종료 코드 2/3)은 명세에 정의되지 않아 AI가 제안하고 팀이 채택한 것이다. 이때 작업 디렉터리에 **이전 실행의 data.out이 있으면 그대로 남는다**(삭제하지 않음). 따라서 data.out은 종료 코드가 0일 때만 이번 실행의 결과다. 채점 입력은 항상 정상 종료하므로(명세 6절) 영향이 없다.
- **실행 방식:** 명세대로 별도 프로세스(`python3 sim.py ...`)로 실행하는 것을 전제로 한다. 표준 출력은 `sys.stdout.buffer`(바이트 스트림)로 쓰므로, 다른 파이썬 코드가 `main()`을 직접 import해 `sys.stdout`을 텍스트 전용 객체로 바꾼 상태에서는 동작하지 않는다.
- **플랫폼 의존 테스트:** 텍스트 모드 줄바꿈 변이는 Windows에서만 검출된다(리눅스에서는 변환이 없어 결과가 같음).
- **Python 버전:** 3.8 이상이 필요하다(3.8.20, 3.10.0, 3.13.18에서 확인).

10. 팀원별 기여 (⑩)

【작성 필요 - 실제 수행 내용을 사실대로 적는다. 아래 역할 구분은 모듈 구조에 따른 예시다.】

팀원	담당	구체적 기여 (파일·함수)	비율
성주원	【예: 명령어 해독·즉시값】	【예: `rvsim/decoder.py` 검토, 즉시값 손 계산 검증】	【%】
김도윤	【예: 실행·분기·종료】	【예: `rvsim/cpu.py`, 실행 한도 테스트 도출】	【%】
송윤재	【예: 메모리·바이너리 입출력】	【예: `rvsim/memory.py`, `sim.py`, Windows/Linux 실행 확인】	【%】
정재운	【예: 테스트·보고서】	【예: `tests/cases.py` 기대 결과 손 검산, 보고서】	【%】

모든 팀원은 담당 외에도 fetch/decode/execute 흐름, 즉시값 부호 확장, little-endian, invalid 판별과 종료 출력의 원리를 설명할 수 있도록 준비했다. 【실제로 한 준비 방법을 적는다】

11. AI 도구 정보·활용 정도 및 검증 (⑪)

11.1 도구 식별과 사용 방식

항목	내용
서비스·제품명	Claude Code (명령줄 AI 코딩 에이전트)
제공사	Anthropic
모델명·버전	Claude Opus 5.5 (모델 ID `claude-opus-5-5`, 도구가 표시한 값)
사용 시기	2026-11-08【추가 사용 날짜가 있으면 기재】
사용 팀원	【사용한 팀원 이름】
사용 방식	에이전트 방식: 명세서 PDF를 읽고 분석한 뒤, 사용자 승인 아래 파일 생성·명령 실행(테스트, Docker)까지 수행
그 밖의 AI 도구	【없으면 "없음". 있으면 도구마다 같은 항목을 표로 추가. 모델이 표시되지 않으면 "확인 불가"】

11.2 목적·대상

명세 분석(요구사항·모호한 점 정리), 언어 선택 조언, 전체 설계, 명령어 해독·실행 코드, 바이너리 입출력, 테스트 설계와 기대 결과 도출, 테스트 도구(어셈블러·실행기·변이·참조 대조), 디버깅 도구, README와 이 보고서 초안 작성.

11.3 영역별 활용표

등급: 0 미사용, 1 보조, 2 부분 생성, 3 주요 생성 (명세 9.2절 정의)

영역	사용 도구	사용자	등급 (근거)	반영 위치	검증 근거
전체 설계	Claude Code	【이름】	3 (모듈 구조·값 표현 원칙·종료 코드를 AI가 제안·작성, 팀은 언어(Python)·CLI 방식을 결정)	모듈 구조(`sim.py`, `rvsim/*`), 값 표현 원칙, 종료 코드 설계	명세 각 절과 대조, 명세 11절 체크리스트【팀 검토 내용】
명령어 해독·즉시값	Claude Code	【이름】	3 (`decoder.py`, `bits.py` 전체를 AI가 생성)	`rvsim/decoder.py` 전체, `rvsim/bits.py`	독립 인코더(`tests/asm.py`) 왕복 시험, 무작위 200,000개 판정 비교, 명세 예시 기계어 대조, Unicorn 대조
산술·논리 연산	Claude Code	【이름】	3 (`CPU.execute`의 연산부 전체를 AI가 생성)	`CPU.execute` (add~lui), `CPU.write_reg`	테스트 03~08 손 계산, 변이 테스트, Unicorn 23,000개 대조
메모리·바이너리 입출력	Claude Code	【이름】	3 (`memory.py`와 출력부 전체를 AI가 생성)	`rvsim/memory.py`, `sim.main` 출력부	테스트 01·02·09~12, data.out 바이트 비교, Windows·Linux 실행
분기·종료	Claude Code	【이름】	3 (`CPU.run`, 분기 실행, 종료 처리를 AI가 생성)	`CPU.run`, `CPU.execute` (beq~bge), `sim.main`	테스트 13~16·22~36 손 계산, 실행 한도 경계 테스트

영역	사용 도구	사용자	등급 (근거)	반영 위치	검증 근거
테스트·디버깅	Claude Code	【이름】	3 (테스트 케이스·도구 전체를 AI가 생성하고 실행)	`tests/` 전체, `tools/trace.py`	변이 테스트 29/29, Unicorn 손 계산 33/33 일치【팀 손 검산 결과】
보고서	Claude Code	【이름】	3 (README와 보고서 초안을 AI가 작성, 팀은 팀 정보를 제공)	README.md, 이 보고서 초안	【팀이 소스와 대조·수정한 내용】

【팀이 직접 코드를 고치거나 다시 작성한 영역이 있으면, 그 영역의 등급과 근거를 실제에 맞게 고친다. 명세 9.2절: 여러 등급이 섞인 영역은 가장 높은 등급을 표시하고 하위 범위를 설명한다.】

11.4 채택·수정 범위

- **채택:** 위 표의 코드·테스트·문서는 AI 에이전트가 생성했고, 에이전트가 테스트·변이 테스트·참조 대조·Docker 실행으로 검증했다. 【팀이 직접 실행·검토한 내용을 기재】
- **AI 작업 중 수정:**
  - 변이 테스트에서 테스트 빈틈 2개가 드러나 테스트 08·34를 추가했다(8장).
  - 추가로 테스트 번호를 범주별로 다시 매겼다.
- **팀의 수정·보완:** 【팀이 직접 바꾼 코드·주석·테스트·문서를 구체적으로 적는다. 없으면 "없음"이라고 쓴다】

11.5 검증

- **독립 기대 결과:** 모든 테스트의 기대값을 실행 결과가 아니라 명령어 의미로 계산하고, 근거를 `tests/cases.py` 주석에 남겼다.
- **독립 인코더:** 테스트 입력은 해독기와 코드를 공유하지 않는 어셈블러로 만들었다. 그 출력은 명세 7절 바이트열과 대조했다.
- **참조 실행기:** Unicorn Engine 2.1.4(QEMU 기반, 우리 팀이 작성하지 않은 독립 구현)를 **테스트에만** 썼다.
  - 정상 종료 케이스 33개의 손 계산 data.out과 무작위 프로그램 23,000개가 모두 일치했다.
  - 시뮬레이터 실행 엔진은 Unicorn을 전혀 쓰지 않는다(명세 8절).
- **변이 테스트:** 흔한 실수 29종이 모두 테스트에 걸리는 것을 확인했다.
- **새 환경:** Docker 리눅스 컨테이너에서 README 명령만으로 실행해 통과했다.
- **팀의 독립 검증:** 【예: 팀원 각자가 테스트 ○개의 기대 결과를 직접 손으로 계산해 일치를 확인, 코드 줄 단위 검토 등 실제로 한 것을 적는다】

11.6 대표 사례

명세 9절에 따라 각 사례에 목적(요청), 제안 내용, 검증 방법, 채택·수정·기각 이유를 적는다.

목적(요청)	AI 제안 내용	검증 방법	처리와 이유
명세서를 분석하고 GUI가 필요한지 확인	10절에 GUI는 가산점이 아니라고 명시되어 있고, 6절(인수 2개)·5.1절(표준 출력 한 줄) 때문에 CLI가 맞다. 명세에 정의되지 않은 동작 3가지(실행 한도 초과 출력, 범위 위반, 크기 위반)도 짚었다.	AI가 명세서 원문 5.1·6·10절을 근거로 제시 【팀이 원문을 직접 대조했다면 기재】	**채택** - 팀이 CLI만 만들기로 결정했고, 미정의 동작은 README·6장에 명시했다.

목적(요청)	AI 제안 내용	검증 방법	처리와 이유
주력 언어 추천	Python 3: 빌드가 필요 없고, 인쇄본이 짧으며, 32비트 처리가 코드에 명시적으로 드러난다. C는 부호 있는 오버플로가 정의되지 않은 동작이라 위험하다.	새 환경(Docker Python 3.8/3.13)에서 README 명령만으로 실행	**채택** - 팀원 모두가 읽을 수 있는 언어로 선택했다.
테스트 충분성 검증 (오류 수정 사례)	변이 테스트를 제안·구현했다. 1차 실행에서 테스트가 놓친 버그 2종(32비트 자르기 누락, 한도 비교 `>`)을 찾아 테스트 08·34를 추가했다.	추가 전 27/29 → 추가 후 29/29 검출	**채택** - 8장 1·2번.
표준 출력·data.out 쓰기 방식	Windows 줄바꿈 변환을 피하려고 바이너리 출력을 쓰자고 했다.	텍스트 모드 변이 2종을 Windows에서 실행해 테스트 실패(검출) 확인	**채택** - 효과를 실제로 확인했다.
속도 개선	해독 결과 캐시를 검토했다.	1,000,000개 실행 시간 측정(약 2.5초)	**채택하지 않음** - 속도는 평가 항목이 아니고, fetch/decode/execute 흐름을 그대로 보이는 쪽이 설명에 유리하다.
【팀의 추가 사례: 수정·기각한 AI 제안, AI 오류를 발견한 경우 등】			

11.7 외부 자료·의존성 출처

자료	용도	범위
프로젝트 명세서	요구사항	전체
The RISC-V Instruction Set Manual, Volume I (RV32I Base Integer ISA)	명령어 형식·opcode·funct3·funct7·즉시값 배치	`decoder.py`, `tests/asm.py`의 인코딩 표
Python 3 표준 라이브러리	파일 입출력, 테스트(unittest, subprocess)	전체
Unicorn Engine 2.1.4 (`pip install unicorn`)	참조 실행기, **테스트 전용**	`tests/diff_ref_unicorn.py`만
Python-Markdown	보고서 PDF 변환	`report/build_report.py`만

부록 A. 자체 테스트 목록

모든 케이스의 파일은 `tests/cases/<이름>/`에 있다. 목적과 도출 근거는 `tests/cases.py`와 `tests/cases/INDEX.md`에 있다.

케이스	기대 표준 출력 / 종료 코드	핵심 기대 data.out
01_spec_example	`PC 0x00040014 invalid instruction 0x00000000`	[0] = 0x0000000c
02_immediate_invalid	`PC 0x00040000 invalid instruction 0x00000000`	data.bin 1,024 B 그대로

케이스	기대 표준 출력 / 종료 코드	핵심 기대 data.out
03_x0_hardwired	`PC 0x00040028 invalid instruction 0x00000000`	[4] = 0, [8] = 7
04_add_sub_wrap	`PC 0x00040040 invalid instruction 0x00000000`	0x80000000, 0xffffffff, 0, 0xfffffffe, 0x7fffffff, 0x80000001
05_addi_imm	`PC 0x0004003c invalid instruction 0x00000000`	0xffffffff, 0xfffff800, 0x7ff, 0x800, 0, 0xfffff000, 0x123
06_logic	`PC 0x0004004c invalid instruction 0x00000000`	0x12005070, 0xff34f6f8, 0x12345678, 0x678, 0xfffffe78, 0x12345600, 0x555
07_lui	`PC 0x00040040 invalid instruction 0x00000000`	0xfffff000, 0x1000, 0x12345fff, 0x80000000, 0xdeadbeef, 0x54321000
08_wrap_compare	`PC 0x0004003c invalid instruction 0x00000000`	[4] = 4
09_lw_sw_endian	`PC 0x0004002c invalid instruction 0x00000000`	[0] = 01 02 03 04, [16] = 78 56 34 12, [1020] = ef be ad de
10_sw_lw_imm_split	`PC 0x00040038 invalid instruction 0x00000000`	[0x1fc], [0x3fc] = 0xa1b2c3d4, [0x3f8] = 0xa1b2c3d5
11_data_empty	`PC 0x00040024 invalid instruction 0x00000000`	0d 00 00 00 / 0d 0a 0d 0a / … / [1020] = 0a
12_data_partial	`PC 0x00040014 invalid instruction 0x00000000`	01..06 00 00 05 06 00 00 01 02 03 04
13_beq_bne	`PC 0x00040038 invalid instruction 0x00000000`	[0] = 0x0a
14_blt_bge_signed	`PC 0x0004005c invalid instruction 0x00000000`	[0] = 0xa6
15_branch_far_offsets	`PC 0x00040014 invalid instruction 0x00000000`	[0] = 6
16_branch_over_invalid	`PC 0x00040018 invalid instruction 0x0000006f`	[0] = 1
17_loop_sum	`PC 0x00040028 invalid instruction 0x00000000`	[64] = 55
18_fibonacci	`PC 0x0004002c invalid instruction 0x00000000`	F(0)~F(19)
19_multiply_signed	`PC 0x00040020 invalid instruction 0x00000000`	[8] = 0xffffffeb (-21)
20_bubble_sort	`PC 0x00040038 invalid instruction 0x00000000`	-7, -3, -1, 0, 1, 5, 8, 12
21_all_registers	`PC 0x000400f8 invalid instruction 0x00000000`	[4i] = i, [124] = 0x10000000
22_invalid_mul	`PC 0x00040014 invalid instruction 0x023102b3`	[0] = 7, [4] 변경 없음
23_invalid_sra	`PC 0x00040014 invalid instruction 0x403152b3`	〃

케이스	기대 표준 출력 / 종료 코드	핵심 기대 data.out
24_invalid_and_funct7	`PC 0x00040014 invalid instruction 0x403172b3`	〃
25_invalid_xor	`PC 0x00040014 invalid instruction 0x003142b3`	〃
26_invalid_slli	`PC 0x00040014 invalid instruction 0x00311293`	〃
27_invalid_lb	`PC 0x00040014 invalid instruction 0x00008283`	〃
28_invalid_sh	`PC 0x00040014 invalid instruction 0x00209223`	〃
29_invalid_bltu	`PC 0x00040014 invalid instruction 0x00316463`	〃
30_invalid_jal	`PC 0x00040014 invalid instruction 0x0000006f`	〃
31_invalid_ecall	`PC 0x00040014 invalid instruction 0x00000073`	〃
32_invalid_all_ones	`PC 0x00040014 invalid instruction 0xffffffff`	〃
33_limit_exact	`PC 0x00040010 invalid instruction 0x00000000`	전부 0
34_limit_plus_one	표준 출력 없음, 종료 코드 2	-
35_limit_exceeded	표준 출력 없음, 종료 코드 2	-
36_infinite_loop	표준 출력 없음, 종료 코드 2	-
37_err_lw_out_of_range	표준 출력 없음, 종료 코드 3	-
38_err_sw_misaligned	표준 출력 없음, 종료 코드 3	-
39_err_pc_out_of_text	표준 출력 없음, 종료 코드 3	-
40_err_sw_to_text	표준 출력 없음, 종료 코드 3	-

부록 B. 변이 테스트 결과

심은 버그	파일	실패한 테스트 수	실패한 테스트(일부)
blt 를 부호 없는 비교로 구현	rvsim/cpu.py	1	14_blt_bge_signed
bge 를 부호 없는 비교로 구현	rvsim/cpu.py	2	14_blt_bge_signed, 20_bubble_sort
bne 조건 반전	rvsim/cpu.py	7	08_wrap_compare, 13_beq_bne, 17_loop_sum, 18_fibonacci 외 3개
분기 목표를 PC+4 기준으로 계산	rvsim/cpu.py	10	08_wrap_compare, 14_blt_bge_signed, 15_branch_far_offsets, 16_branch_over_invalid 외 6개
x0 쓰기 보호 누락	rvsim/cpu.py	1	03_x0_hardwired

심은 버그	파일	실패한 테스트 수	실패한 테스트(일부)
레지스터 값을 32비트로 자르지 않음	rvsim/cpu.py	1	08_wrap_compare
andi/ori 즉시값을 부호 확장하지 않음(하위 12비트만)	rvsim/cpu.py	1	06_logic
lui 값을 한 번 더 12비트 시프트	rvsim/cpu.py	32	01_spec_example, 03_x0_hardwired, 04_add_sub_wrap, 05_addi_imm 외 28개
lw 에서 오프셋 무시	rvsim/cpu.py	6	09_lw_sw_endian, 10_sw_lw_imm_split, 12_data_partial, 19_multiply_signed 외 2개
sw 가 rs2 대신 rs1 을 저장	rvsim/cpu.py	31	01_spec_example, 03_x0_hardwired, 04_add_sub_wrap, 05_addi_imm 외 27개
종료 시 fetch PC 대신 PC+4 출력	rvsim/cpu.py	33	01_spec_example, 02_immediate_invalid, 03_x0_hardwired, 04_add_sub_wrap 외 29개
실행 한도 비교를 > 로 (1,000,001 개 허용)	rvsim/cpu.py	1	34_limit_plus_one
실행 한도 검사를 invalid 판별보다 먼저 수행	rvsim/cpu.py	1	33_limit_exact
I형 즉시값 부호 확장 누락	rvsim/decoder.py	14	04_add_sub_wrap, 05_addi_imm, 06_logic, 07_lui 외 10개
S형 즉시값 imm[4:0] 누락	rvsim/decoder.py	14	03_x0_hardwired, 04_add_sub_wrap, 05_addi_imm, 06_logic 외 10개
B형 즉시값 imm11 누락	rvsim/decoder.py	8	15_branch_far_offsets, 17_loop_sum, 18_fibonacci, 19_multiply_signed 외 4개
B형 즉시값을 12비트로 부호 확장	rvsim/decoder.py	1	15_branch_far_offsets
R형 funct7 검사 누락 (sub 외에는 add 로 간주)	rvsim/decoder.py	1	22_invalid_mul
sub 를 add 로 해독	rvsim/decoder.py	1	04_add_sub_wrap
lw 의 funct3 검사 누락	rvsim/decoder.py	1	27_invalid_lb
sw 의 funct3 검사 누락	rvsim/decoder.py	1	28_invalid_sh
I형 연산 funct3 검사 누락 (모르는 funct3 를 addi 로)	rvsim/decoder.py	1	26_invalid_slli
bltu/bgeu 를 blt/bge 로 해독	rvsim/decoder.py	1	29_invalid_bltu
little-endian 대신 big-endian 으로 읽기	rvsim/memory.py	39	01_spec_example, 03_x0_hardwired, 04_add_sub_wrap, 05_addi_imm 외 35개
data.out 을 텍스트 모드로 저장	rvsim/memory.py	6	02_immediate_invalid, 11_data_empty, 13_beq_bne, 17_loop_sum 외 2개
data.out 을 1,020 바이트만 저장	rvsim/memory.py	33	01_spec_example, 02_immediate_invalid, 03_x0_hardwired, 04_add_sub_wrap 외 29개
표준 출력을 텍스트 모드 print 로 출력	sim.py	33	01_spec_example, 02_immediate_invalid, 03_x0_hardwired, 04_add_sub_wrap 외 29개
16진수를 대문자로 출력	sim.py	14	05_addi_imm, 06_logic, 08_wrap_compare, 09_lw_sw_endian 외 10개

심은 버그	파일	실패한 테스트 수	실패한 테스트(일부)
16진수 선행 0 생략	sim.py	33	01_spec_example, 02_immediate_invalid, 03_x0_hardwired, 04_add_sub_wrap 외 29개

부록 C. 팀 사진

【프로젝트 실행 중 팀원 전원이 함께 찍은 사진을 report/team_photo.jpg 로 저장한 뒤 다시 빌드】